Circular Doubly Linked List (CDLL)

What is a Circular Doubly Linked List?

A Circular Doubly Linked List is a type of linked list in which:

- Each node contains **two pointers**:
  - One to the **next node**
  - One to the **previous node**
- The list is **circular**, meaning:
  - The **last node points to the first node**
  - The **first node points back to the last node**

Unlike a normal doubly linked list, there is **no NULL pointer** at the ends.

Structure of a Node

Each node consists of:

- **Prev Pointer** – points to previous node
- **Data** – stores the value
- **Next Pointer** – points to next node

Representation:

```
[ Prev | Data | Next ]
```

Example CDLL:

```
      ┌──────────────────────────────┐
      ↓                              │
[30] ⇄ [10] ⇄ [20] ⇄ [30] ⇄ [10] ...
      ↑                              │
      └──────────────────────────────┘
```

Or simply:

```
Head → [10] ⇄ [20] ⇄ [30]
 ↑                        ↓
 └──────────── Circular ────────┘
```

Declaration of Node in C

```
struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
```

Head Pointer

```
struct Node* head;
```

- `head` points to the first node
- `head->prev` points to the last node
- Last node's `next` points to head

Characteristics of CDLL

- Nodes are connected in a **circular manner**
- Each node has **two pointers (prev and next)**
- No node contains NULL
- Traversal can start from **any node**
- Supports **bidirectional traversal**

Advantages of CDLL

1. Bidirectional Traversal

Can move forward and backward easily.

2. Circular Nature

No need to check for NULL during traversal.

3. Efficient Insertion/Deletion

Operations are efficient at both ends.

4. Useful for Repetitive Processes

Ideal for applications that loop continuously.

5. Better Navigation

Easy to move to last node using `head->prev`.

Disadvantages of CDLL

1. Complex Structure

More complex than singly and doubly linked lists.

2. Extra Memory Usage

Two pointers per node increase memory requirement.

3. Careful Handling Required

Wrong pointer updates can break the circular structure.

Operations on CDLL

- Creation
- Insertion
  - At Beginning
  - At End
  - At Position
- Deletion
  - At Beginning
  - At End
  - At Position
- Traversal (Forward & Backward)

Real-Life Use Cases of CDLL

1. Music Playlist (Loop Mode)

Songs repeat continuously in a loop.

2. Round Robin Scheduling

Used in CPU scheduling algorithms.

3. Multiplayer Games

Turn-based systems rotate between players.

4. Navigation Systems

Continuous navigation without end.

5. Buffer Management

Used in circular buffers.

Key Points to Remember

- CDLL is a **doubly linked list with circular connection**
- No node has NULL pointer
- `head->prev` points to last node
- Last node's `next` points to head
- Traversal is possible in **both directions**
- Requires careful pointer handling